

# Unity+GitHub项目管理通用技术手册

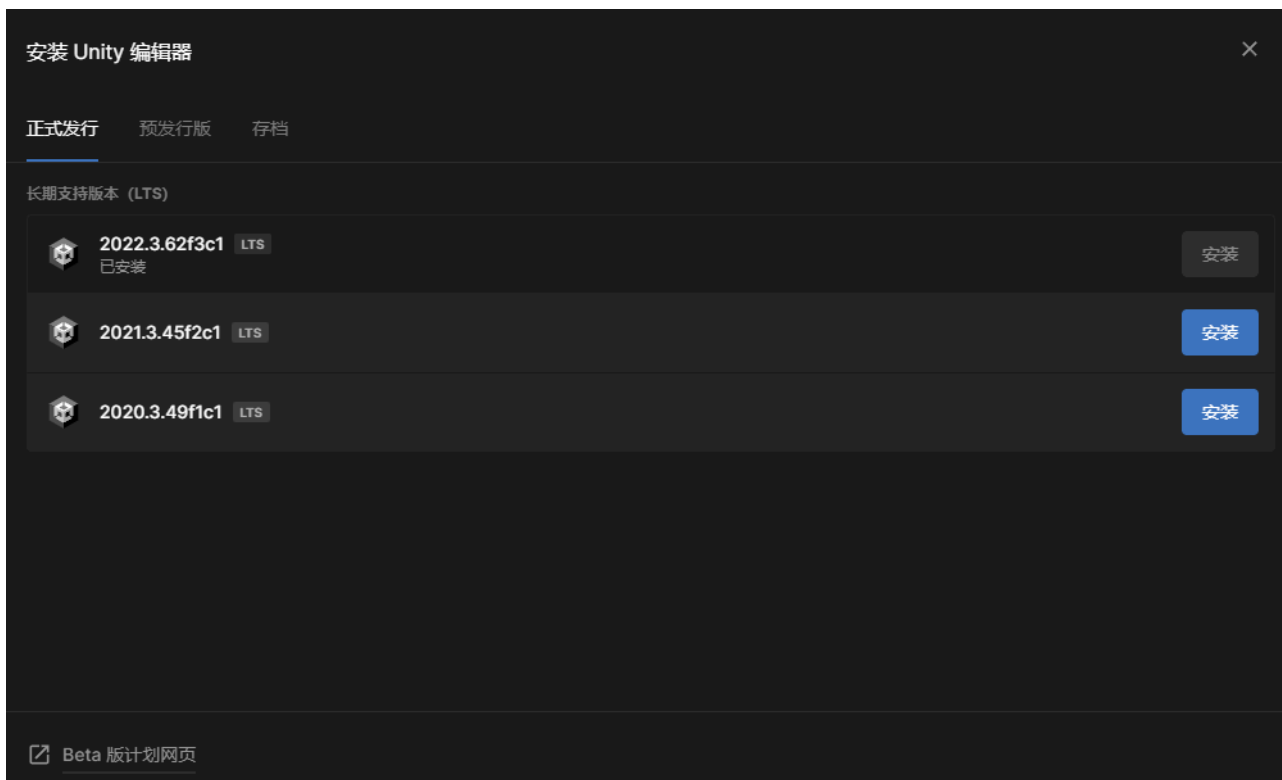
## 一、前期准备

### 1. Unity版本

Unity引擎具有多个版本，如 `2022.3.62f3c1`，`2021.3.45f2c1` 等，请确保项目组的所有人使用的Unity版本一致，推荐使用长期支持版本，当前Unity国内最新的长期支持版本为 `2022.3.62f3c1`。

最大可接受的版本差异为版本号最后的c1c2，如 `2022.3.62f2c1` 和 `2022.3.62f2c2` 是可以兼容的，但 `2022.3.62f2c1` 和 `2022.3.62f3c1` 是存在兼容问题的。不过笔者强烈推荐同一项目的Unity小版本号也不要有什么区别。

Unity 从 Unity 6 开始改了版本发布策略，采用“大版本 + 更新版本 + 补丁”的三层架构。截至 2026 年，Unity 6.3 LTS 是当前的长期支持版本，Unity 6.0 LTS 继续支持到 2026 年 10 月。Unity 6 系列彻底抛弃了以前“2020/2021/2022”这种年份命名，改用 `6000.x` 的编号体系。



### 2. Git LFS

GitHub 对单个文件的限制是 100MB，超过这个大小的文件直接拒绝推送。Unity 项目的美术资源，比如高分辨率贴图、3D 模型、音频文件，很容易超过这个限制。Git LFS (Large File Storage) 就是专门解决这个问题的——它把这些大文件存到外部，Git 仓库里只保留一个轻量级的指针。

安装和配置也很简单：

代码块

```
1 # 安装 Git LFS (每台机器装一次就行)
2 git lfs install
3
4 # 告诉 LFS 跟踪哪些类型的文件
5 git lfs track "*.psd" "*.fbx" "*.wav" "*.mp3" "*.tga" "*.png"
```

执行后会生成或更新 `.gitattributes` 文件，记得把这个文件也提交上去。

有个坑要注意：Git LFS 一定要在项目最开始就配好。如果项目做了一半才加 LFS，之前已经提交的大文件需要手动迁移，非常麻烦。

另外 GitHub Desktop 虽然会自动安装 Git LFS，但不会自动生成 `.gitattributes` 文件，所以你还是得手动配置。

### 3. Unity编辑器设置（非必须，一般这种问题交给本地Agent即可）

虽然较新的Unity版本默认已经设置好了，但为了防止万一，必须要求所有成员检查：

Edit > Project Settings > Editor

在这个面板里，你会看到两个独立的设置区域：

- Version Control 区域：设置 Mode 为 Visible Meta Files（让 `.meta` 文件可见）
- Asset Serialization 区域：设置 Mode 为 Force Text（强制文本序列化）

*原因说明：* 如果不开启 Force Text，Unity 的场景(Scene)和预制体(Prefab)会以二进制保存，一旦发生冲突，Git 根本无法合并，只能覆盖。开启后是以文本（YAML）保存的，可以在代码编辑器里对比冲突。

即使开启了 Force Text，Unity 的 Scene 和 Prefab 文件在 Git 合并时仍然可能出现问题，因为 Git 默认的合并工具不理解 Unity 的 YAML 结构。你需要额外配置 UnityYAMLMerge（也叫 Smart Merge），它专门用来以语义正确的方式合并 Unity 的场景和预制体文件。

配置方法很简单，在你的 `.gitconfig` 文件末尾加上：

代码块

```
1 [merge]
2     tool = unityyamlmerge
3 [mergetool "unityyamlmerge"]
4     trustExitCode = false
5     cmd = '<UnityYAMLMerge的路径>' merge -p "$BASE" "$REMOTE" "$LOCAL" "$MERGED"
```

Windows 下 UnityYAMLMerge 的默认路径通常是：

```
C:\Program Files\Unity\Editor\Data\Tools\UnityYAMLMerge.exe
```

Mac 下则是：

```
/Applications/Unity/Unity.app/Contents/Helpers/UnityYAMLMerge
```

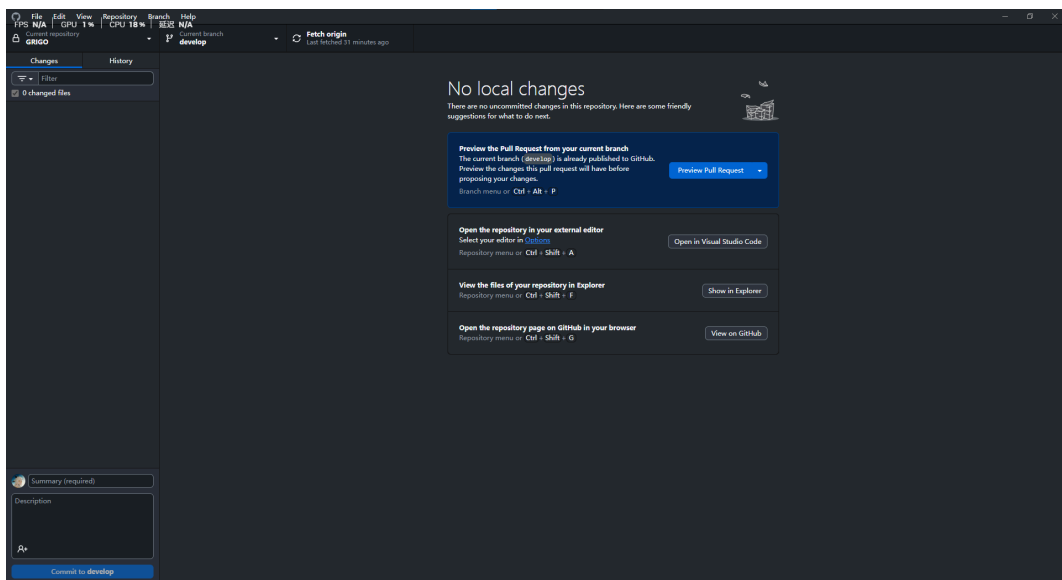
然后在 `.gitattributes` 文件里告诉 Git，`.unity` 和 `.prefab` 文件走这个合并工具：

```
.unity merge=unityyamlmerge.prefab merge=unityyamlmerge
```

配好之后，Unity 场景冲突不再是“地狱级”了，至少能用语义合并来抢救一下。

## 4. git管理工具

使用GitHub仓库来管理Unity项目需要上传和下拉更新，也经常需要切换分支，虽然直接Git Bash也能使用，但是不太方便。此时就需要一个好用的管理工具来提升效率，这里推荐 `GitHub Desktop`，简洁好用（具体使用说明见后文），当然也有很多其他的管理工具如 `SourceTree`，`Fork` 等。部分IDE也集成了git管理功能，如VS，VSC。



## 5. Unity 6 的不同之处

如果你从 Unity 2022 升级到 Unity 6，有几个跟版本控制相关的点需要留意。

### 5.1 版本号格式变了

以前是 `2022.3.62f1c1` 这种格式，Unity 6 改用 `6000.x` 系列。比如：

- Unity 6.0 LTS → `6000.0`
- Unity 6.3 LTS → `6000.3`
- Unity 6.1 更新版 → `6000.1`

Unity 6.3 LTS 的支持周期到 2027 年 12 月，Unity 6.0 LTS 到 2026 年 10 月。选版本的时候可以参考这个来规划项目周期。

### 5.2 发布节奏不一样了

Unity 6 采用 “LTS + 更新版本” 双轨制：

- LTS 版本：比如 6.0 LTS、6.3 LTS，适合需要长期稳定运行的项目，建议游戏上线后锁死在某个 LTS 版本上
- 更新版本：比如 6.1、6.2，包含最新功能和对新平台的支持，同样经过完整测试，适合正在开发中的项目

简单说就是：新项目可以用最新的更新版本，已经上线的项目老老实实待在 LTS 上别乱动。

## 5.3 Unity Hub 集成了 GitHub 和 GitLab

Unity 6 的 Hub 在创建新项目时，可以直接选择 GitHub 或 GitLab 作为源码管理提供商，自动帮你创建远程仓库并连接。不过说实话，对于团队协作来说，我仍然推荐先手动建好仓库再克隆到本地，这样目录结构更可控，不容易出现后面说的“失败方案二”。

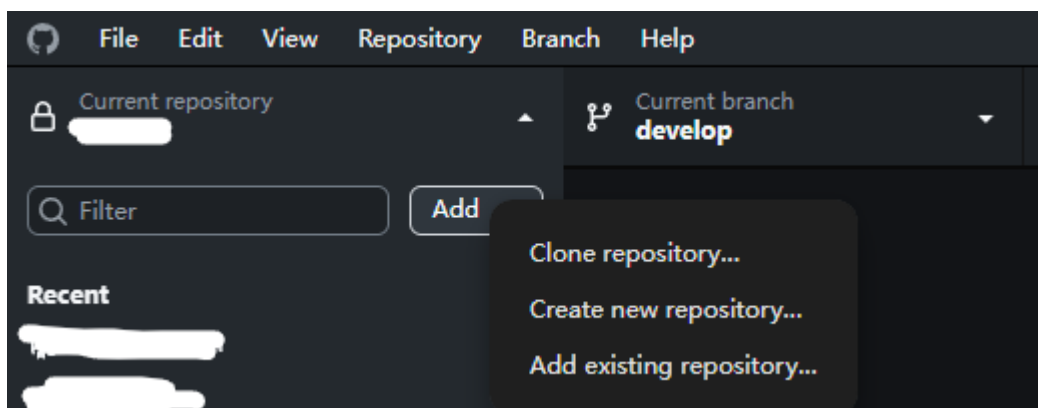
## 二、项目建立

### 1. 建立GitHub仓库

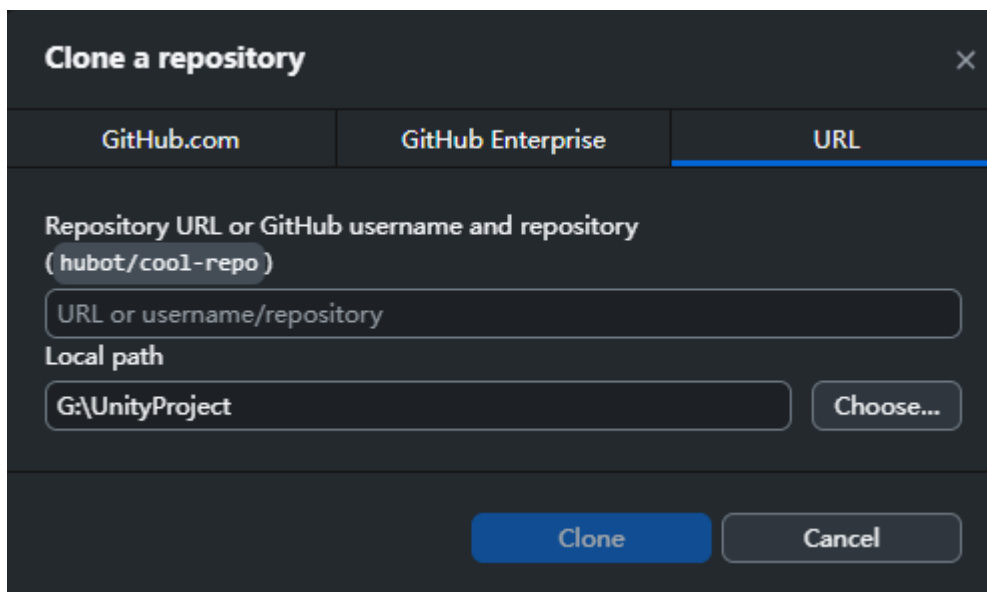
在GitHub上新建一个仓库，在 `.gitignore` 一栏搜索Unity并添加。

### 2. 克隆GitHub仓库到本地

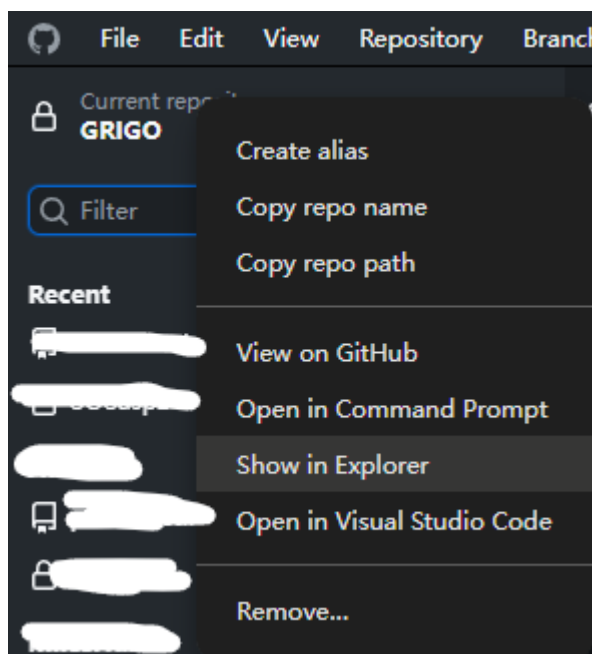
复制仓库网址。本地打开 `GitHub Desktop`，左上角 `add` → `Clone repository`，选择 URL，粘贴刚才复制的网址（如果这里你在 `GitHub Desktop` 上登陆了GitHub，那么可以直接在 [GitHub.com](https://github.com) 一栏选择自己创建或者有权限的分支克隆）。



修改项目要放置的本地地址（当然你懒得改也没问题），然后点击Clone即可。

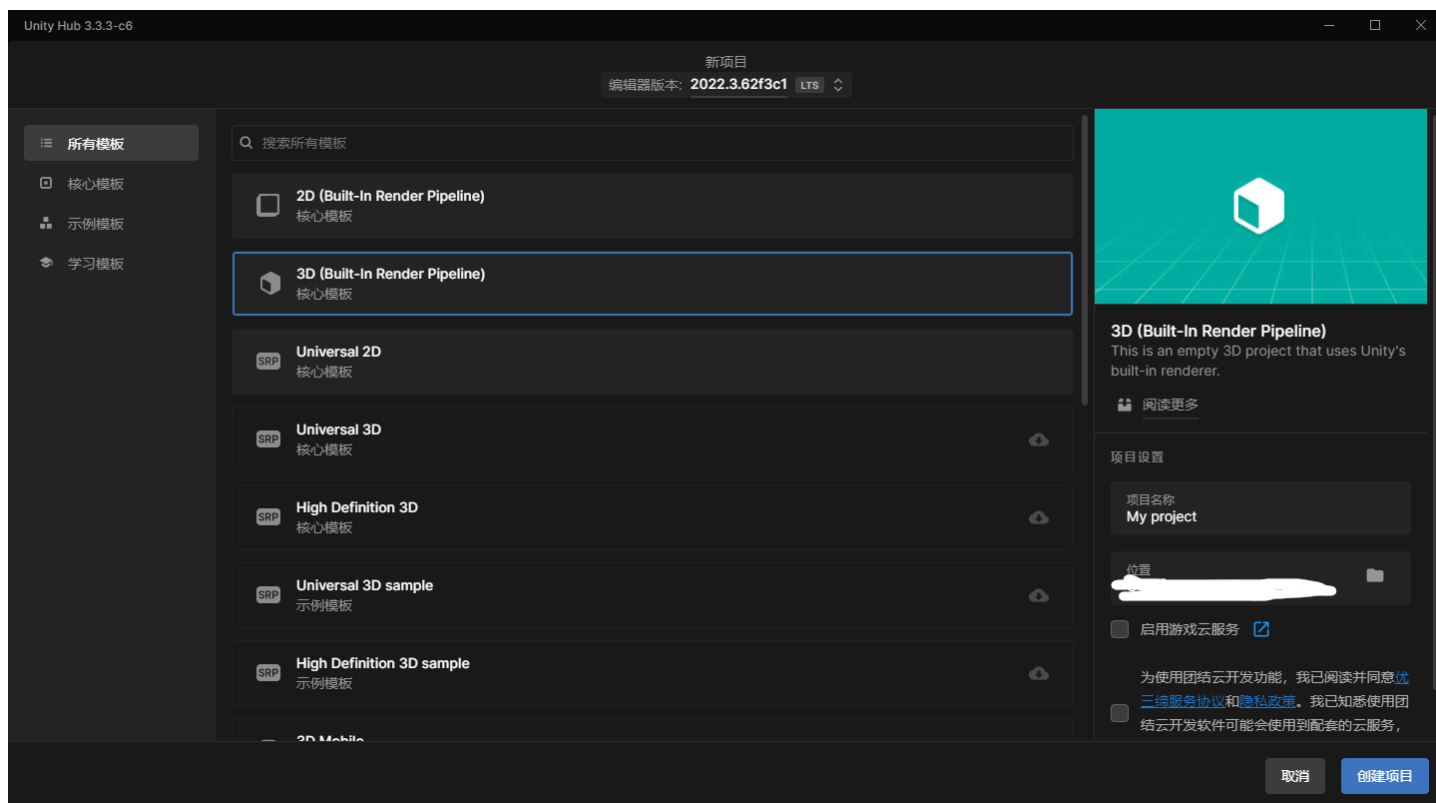


记住你本地仓库的位置，或者右键 `Show in Explorer` 打开项目目录。



### 3. 新建Unity项目

打开Unity Hub，创建新项目，选择自己需要的模板类型。注意上方的编辑器版本，确保是你们项目组之间沟通确认好的版本。选择的项目位置建议直接选择在前面git本地仓库下。等待项目创建完成，关闭Unity项目窗口。



## 4. 移动Unity项目到本地仓库

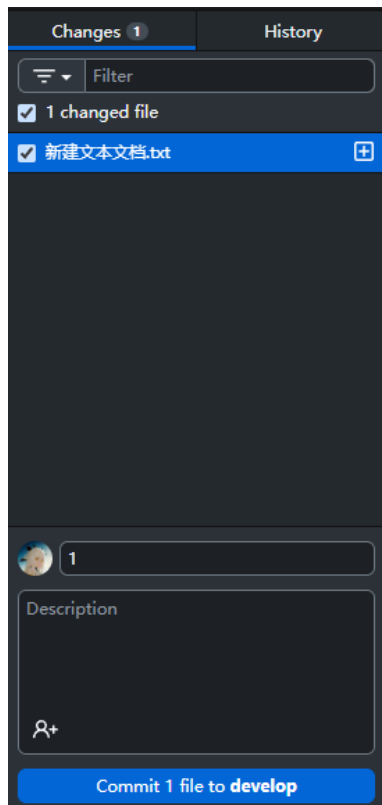
打开Unity项目的文件夹（不是打开项目，是打开存储位置），找到其中 `Assets`，`Packages`，`ProjectSettings`，将这三个文件夹复制到git本地仓库下，删除Unity项目。

（这里有个偷懒的方法，把整个项目的所有文件都拖到本地仓库）

## 5. 上传Unity项目

现在，你应该能看到 `GitHub Desktop` 上多出来了几个文件和文件夹，左下角给这次提交命名，Description可填可不填，就是注释。点击 `Commit`，上方原先 `Fetch origin` 的位置应该变为了 `Push`，点击 `Push` 即可。

注意：如果你前面没有额外加什么东西，那么你需要上传的文件应该很少，肯定不会是几万号文件，如果是几万号文件，说明你是按照下一条注意事项中的失败方案二来处理的，你应当把你项目一级目录中所有的文件和文件夹都拖出来到仓库文件夹下，然后删除原先的项目文件夹。



## 6. 注意事项

我知道很多人觉得上述操作很麻烦，有很多其他的想法，所以这里列举出笔者本人踩过的几个坑。

### a. 失败方案一：

先建本地Unity项目，后克隆git仓库。如果你选择接受git的修改，那么克隆下来的仓库会直接覆盖你的Unity项目，项目内容一点不剩。

### b. 失败方案二：

先克隆git仓库，后在仓库下建Unity。Unity项目会成为仓库的子文件夹，`.gitignore` 无法生效，你的GitHub Desktop会显示有上万个文件要你上传。

### c. 失败方案三：

先克隆，后在仓库上级目录下建Unity。要么存在同名目录，无法新建项目，要么会新建另一个文件夹。

## 三、项目管理

### 1. 代码编写原则/系统划分原则

高内聚、低耦合。

单一脚本尽量仅完成一个功能或者系统，不要把所有东西都堆在一起，既能减少定位bug的难度，也能减少后续合并分支冲突的可能。

两个系统间的不要互相插手（A在B中查找×，A让B自己查找√），区分系统实现方式和玩家可见方式（玩家看到的：A从B的口袋里搜出了东西；系统实际的：A让B检索自己有没有东西并给出）。

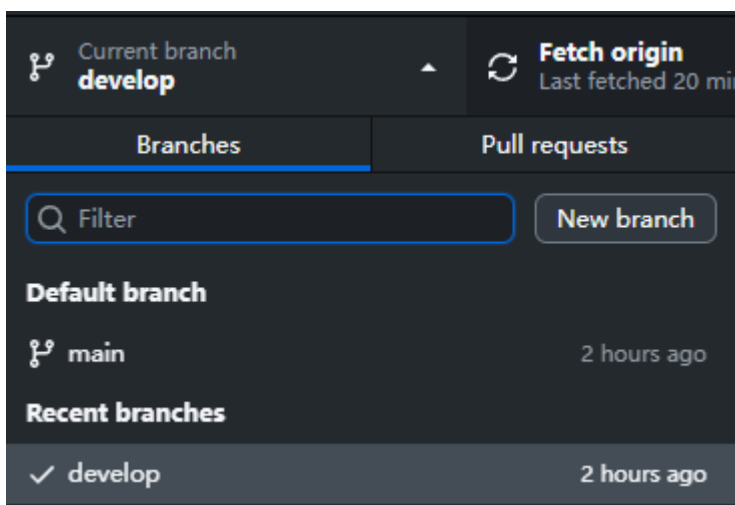
## 2. 避免冲突发生（AI补充）

代码冲突好解决，但Unity的Scene和Prefab冲突是“地狱级”的。

- **不共用场景原则**：绝对避免两个人同时修改同一个Scene。如果需要，可以通过“多场景叠加（Multi-Scene Editing）”的方式，每个人在一个独立的Sub-Scene里工作。
- **预制体化（Prefab）**：把UI、人物、机关全部做成Prefab。开发时大家各自修改自己的Prefab，最后在主场景里只做引用。

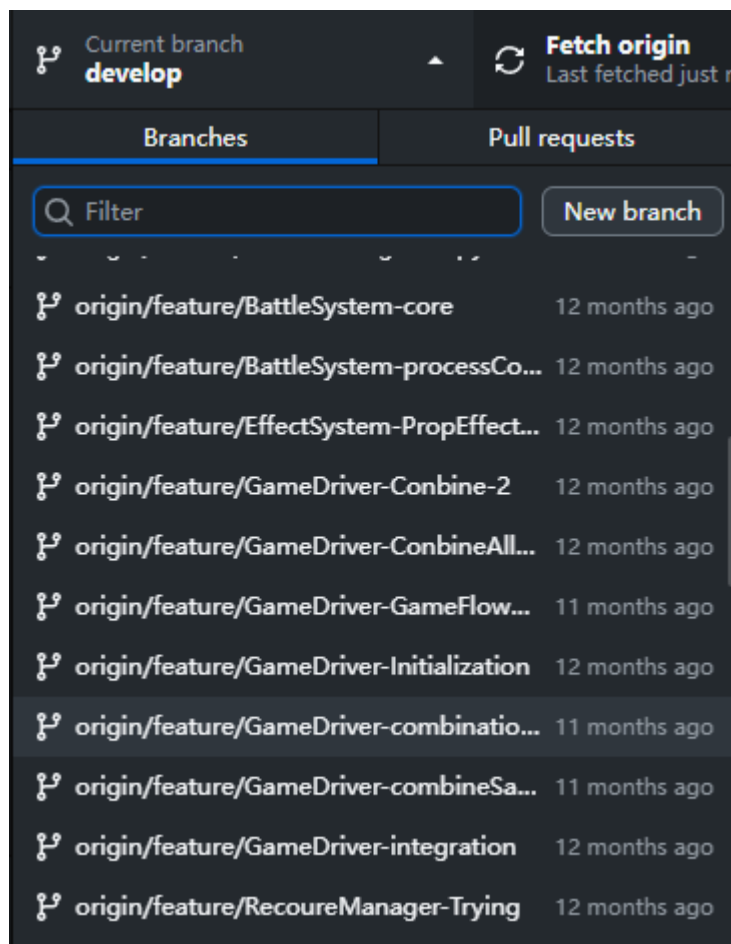
## 3. 分支管理

合理的分支能够让更多的人同时工作，且能减少一人出错全员停滞的情况。



GitHub仓库默认存在一个main分支，推荐在main的基础上分一个develop分支，再在develop的基础上，每一个功能或者系统的开发都拉一个分支，并区分好类型，如feature/GameManager-SceneSwitch, feature/LevelManger-LevelState, test/BattleSystem-SoldierCreate。

（注：你会看到部分分支前带有origin字样表示这是远程仓库中你没有拉取过的分支。feature和test是我个人习惯，标注该分支用于测试还是功能开发）

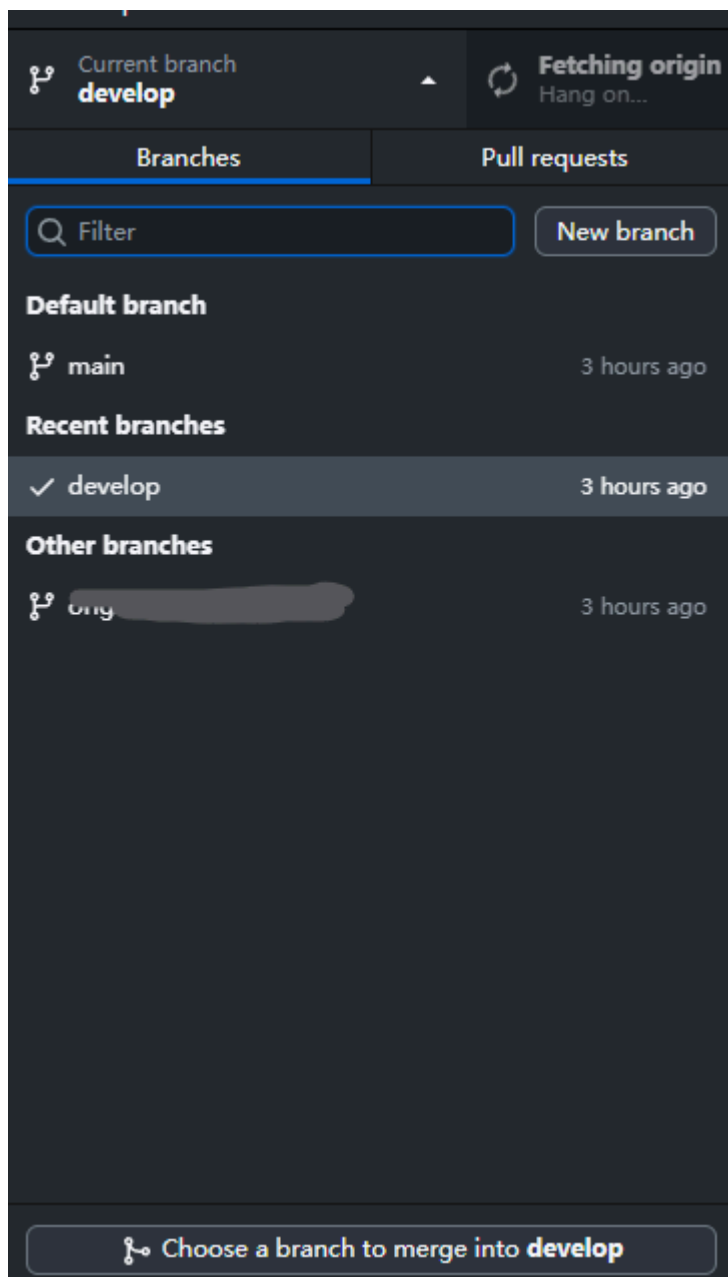


main分支主要用于保存你的可打包版本，develop分支主要用于汇总所有人目前的开发进度，其他的分支就是开发的部分了。

## 4. 分支合并

这里是项目管理重灾区，当你开发好一个功能或者系统后，想要将自己的工作与develop进行合并时，建议按照以下步骤：

- a. 将develop分支与自己的分支合并，检查是否存在冲突。底部Choose a branch to merge into \*\*\*，然后选择develop，选择create a merge commit。



- b. 若存在冲突，记录下系统提示的冲突文件的文件名。放弃合并，切换到develop分支，查看 history，搜索刚才记录下的冲突文件，确认是谁提交的，跟他爆子讨论确认具体的冲突。如果仅仅是修改了同一个文件的不同部分，那么就复制好合并之后再加上就行；如果是修改了同一个文件的同一个部分，依旧爆子那就得对比看看保留谁的了。
- c. 若不存在冲突或者冲突已解决，切换到develop分支，将刚才的分支合并到develop上。
- d. 合并完成后记得删除原分支。

## 5. 每天开始和结束必须做的工作

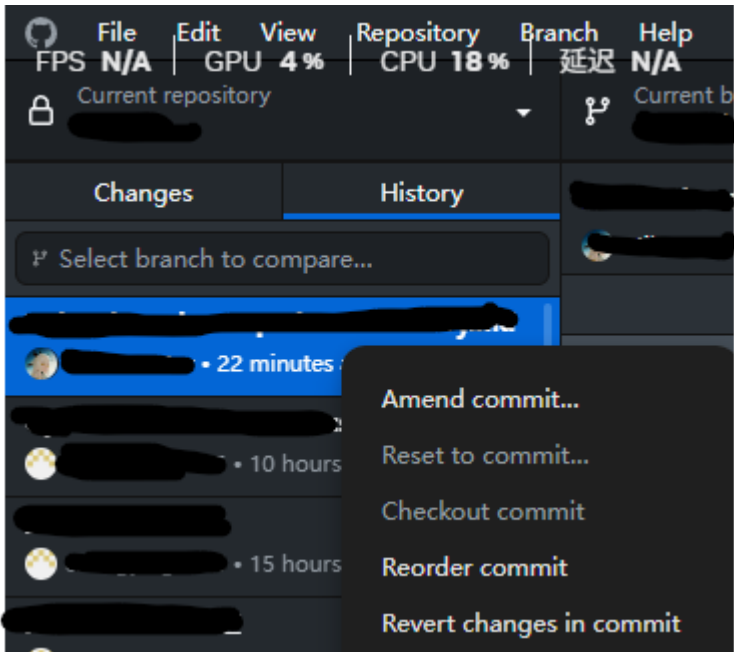
如果你严格按照前面的内容划分好了工作且每个人都只处理一个分支，那么每一位开发人员都需要在每天开始工作前将develop的分支合并到自己的分支上，确保整体最新工作进度不会与个人工作进度产生冲突。如果多个人在同一个分支工作，那么除了从develop分支合并，还需要定时从自己的分支拉取，不要只在自己需要提交时才拉取。

每天结束工作的时候，无论你的代码是否能跑通，至少先提交一次，不用合并到develop，只需要提交到自己的分支即可。这不仅是防止工作内容丢失，也方便出现问题时能最小程度回退。（最好是

和拉取一样定时提交，个人建议每小时拉取和提交一次）

## 6. 版本回退

有些时候我们会遇到某一次提交的内容需要大规模回退的情况（N版方案之后发现第一版更好/某次提交有严重bug不能改只能删），这时候就需要用到“版本回退”（Revert）



Revert是将单一提交涉及到的操作撤销，但是是撤销指定的一次提交。例如我近期依次做了A、B两个提交，我的同事做了了C、D两个提交，当我的同事提交D时发现与A产生了冲突，经商讨后我们决定直接回退A的操作，那么在History一栏直接右键A分支Revert，就能单独撤销A提交，后续提交的B、C不会直接受到影响（假如你的B提交中用到了A提交的内容，那得特殊情况特殊处理）。

## 四、Git 常用控制台命令速查表

下面按场景整理了一套常用命令，如果有些时候版本管理工具卡了或者不够处理你的问题了，可以试试。

### 基础操作

| 命令              | 作用       |
|-----------------|----------|
| git init        | 初始化仓库    |
| git clone <url> | 克隆远程仓库   |
| git status      | 查看当前状态   |
| git add <file>  | 添加文件到暂存区 |
| git add .       | 添加所有修改   |
|                 |          |

|                         |          |
|-------------------------|----------|
| git commit -m "message" | 提交并附上说明  |
| git log --oneline       | 简洁查看提交历史 |

## 分支管理

| 命令                   | 作用                      |
|----------------------|-------------------------|
| git branch           | 查看本地分支                  |
| git branch <name>    | 创建新分支                   |
| git switch <name>    | 切换分支（推荐，比 checkout 更清晰） |
| git switch -c <name> | 创建并切换分支                 |
| git merge <branch>   | 合并指定分支到当前分支             |
| git branch -d <name> | 删除已合并分支                 |
| git branch -D <name> | 强制删除分支                  |

## 远程操作

| 命令                          | 作用         |
|-----------------------------|------------|
| git remote -v               | 查看远程仓库地址   |
| git pull                    | 拉取远程更新并合并  |
| git fetch                   | 拉取远程更新但不合并 |
| git push                    | 推送当前分支     |
| git push -u origin <branch> | 推送并设置上游分支  |

## 撤销与回退

| 命令                        | 作用        | 使用场景     |
|---------------------------|-----------|----------|
| git restore <file>        | 撤销工作区文件修改 | 改错了想恢复   |
| git reset HEAD <file>     | 从暂存区移回工作区 | add 错了文件 |
| git reset --soft <commit> | 回退但保留改动   | 想重新提交    |

|                           |             |            |
|---------------------------|-------------|------------|
| git reset --hard <commit> | 回退并清除所有改动   | 彻底放弃当前修改   |
| git revert <commit>       | 生成新提交撤销指定提交 | 已推送的提交需要撤销 |
| git reflog                | 查看所有操作记录    | 误操作后找回     |

## 临时保存

| 命令             | 作用      |
|----------------|---------|
| git stash      | 暂存当前修改  |
| git stash pop  | 恢复并删除暂存 |
| git stash list | 查看暂存列表  |

以上大部分内容在各个管理工具中都已经集成，放在这里也是以备不时之需。

## 五、项目管理工具推荐（长期更新，本文档可评论，开发者们可以在评论中补充你觉得好用的工具）

1. Lazygit（由 [创游工坊 - ATMLink](#) 推荐：“补一个给开发的推荐工具lazygit，一个开源tui的git工具”）

[Lazygit: 从0到熟练使用，你需要的都在这里](#)

2. SVN（由 [创游工坊 - 祈年](#) 推荐：“美术资源感觉用svn更好”）

[TortoiseSVN安装使用教程（超详细）](#)